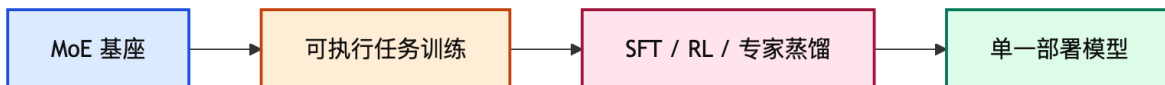


Anchor Note: Qwen3-Coder-Next Coding Agent 训练系统

Coding Agent Notes (June 2026)

1 核心图景

Qwen3-Coder-Next 的核心设计不是单纯扩大总参数，而是把一个稀疏激活模型放进大规模可执行任务训练系统里，让模型从真实环境反馈中学习长期编码行为。



最重要的数字有四个：模型总参数约 80B，但每次前向只激活约 3B；上下文长度扩展到 262,144 tokens；仓库级代码数据约 600B tokens；训练数据中包含约 800K 个可验证软件工程任务。

因此，整个系统的主线是：如果任务能被执行、能被验证、能大规模 rollout，那么 coding agent 的能力可以通过环境反馈持续提升，而不必只依赖静态代码语料或单纯扩大模型。

1.1 系统设计的三条因果链

可以把 Qwen3-Coder-Next 拆成三条相互支撑的因果链：

容量到成本。 Qwen3-Coder-Next 使用 80B total / 3B active 的 MoE，把模型总容量和单次推理成本拆开。对 coding agent 来说，这很关键：真实任务会产生长上下文和多轮工具调用，低延迟、低成本本身就是可部署能力。

静态数据到环境反馈。 训练重点不只是更多代码语料，而是可执行 Docker 任务、测试、rollout 和 RL。真实开发不是补全一个函数，而是在仓库中定位问题、编辑文件、运行工具、观察失败并迭代修复。

单一技能到鲁棒部署。 模型还要适配多 scaffold、多工具格式和不同任务域，所以系统使用多专家训练，再蒸馏到一个单一部署模型。这里的目标不是单项技能最强，而是让 agent 在不同 IDE/CLI 协议下稳定工作。

Insight. 关键不在于提出一个全新的模型结构，而在于把 coding model 的训练目标从“预测代码文本”改造成“在可验证环境里完成软件工程任务”。这会改变数据、基础设施、RL 奖励和评测方式。

2 模型设计：小激活量的大模型

Qwen3-Coder-Next 建在 Qwen3-Next 之上，使用 hybrid attention 与 Mixture-of-Experts (MoE) 结构。下面用 ‘80A3’ 这类记法表示模型规模，其中 80B 是总参数，3B 是每次推理激活的参数量。

维度	设定	直观含义
总参数	80B	模型容量较大，可存储多域能力
激活参数	3B	每次推理只用少量专家，成本低
上下文	262K tokens	支持仓库级、长交互任务
定位	coding agent	面向工具调用、修 bug、跑测试

这里的关键不是“80B 模型只花 3B 成本就拥有全部能力”。更准确地说，MoE 让不同输入路由到不同专家，从而把总容量和单次计算成本分离。对于 coding agent，这很有价值：真实开发任务常常需要长上下文和多轮交互，推理成本与延迟会直接影响可用性。

2.1 80A3 记法与效率含义

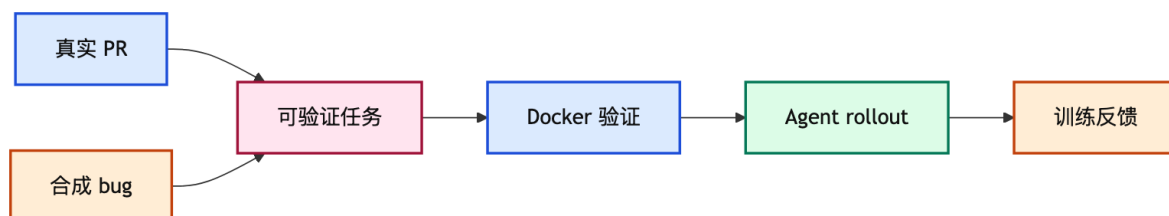
为了比较稀疏模型的效率，size 记法可以读作“总参数 A 激活参数”。例如 DeepSeek-V3.2 的 671A37 表示总参数约 671B、激活参数约 37B；Qwen3-Coder-Next 的 80A3 表示总参数约 80B、激活参数约 3B。在 agent 场景中，active 参数比 total 参数更接近单次推理成本；total 参数更接近模型总容量。

模型	规模	SWE-Bench Pro	效率含义
Qwen3-Coder-Next	80A3	44.3%	小 active compute，强 SWE 表现
DeepSeek-V3.2	671A37	40.9%	active compute 约为 Qwen 的 12×
GLM-4.7	358A32	40.6%	active compute 约为 Qwen 的 10×
MiniMax-M2.1	230A10	34.6%	active compute 约为 Qwen 的 3×

这个比较不是严格的成本基准，因为不同模型的实现、上下文长度、硬件和路由效率不同；但它凸显了 Qwen3-Coder-Next 的关键优势：每个 active 参数带来的 agentic coding 能力很高。

3 Agentic Training: 让任务可执行、可验证

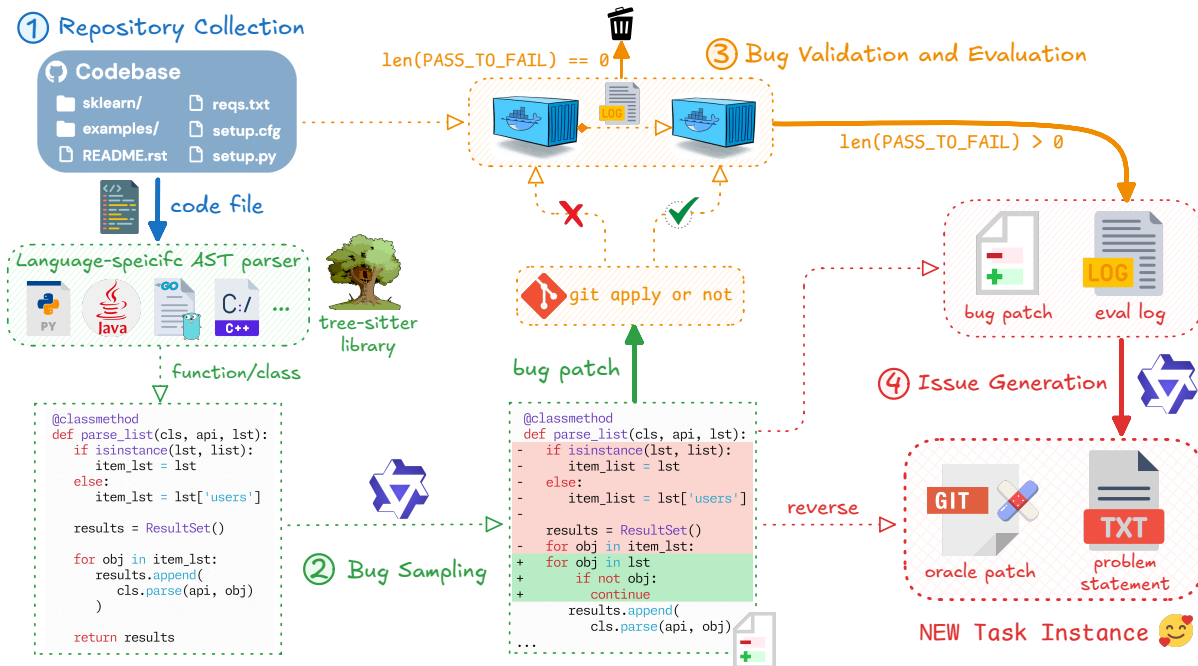
Agentic training 是最核心的工程系统。它要解决两个问题：第一，如何批量制造有标准答案的真实开发任务；第二，如何以足够高吞吐量运行这些任务并收集环境反馈。



3.1 任务合成

任务合成包含两条路线。

- 第一条路线来自真实 GitHub PR：把 PR 拆成 buggy state、fix 和 test patch，再让环境构建 agent 生成可运行 Docker 环境和验证脚本。
- 第二条路线来自已有可执行数据集：在容器化仓库中注入可控 bug，再生成自然语言 issue 描述。保留规则是：bug 必须让测试失败，并且回滚补丁能修复测试。



论文原图：用于扩展软件工程任务数量的 bug synthesis pipeline。

这个流程得到约 800K 个可验证软件工程任务，覆盖九种以上编程语言。它的价值在于把“代码训练”变成“环境内解决问题训练”：模型不只是续写代码，而是要在仓库中定位问题、编辑文件、调用工具、跑测试并修复失败。

3.2 可验证任务的三个不变量

“可验证”不是简单地给任务配一个答案。对软件工程任务来说，至少要满足三个不变量：

不变量	具体要求	为什么重要
失败可观测	buggy state 必须让测试失败	否则任务没有学习信号
修复可验证	oracle patch / patch reversion 能让测试通过	保证任务可解而不是噪声
环境可复现	Docker 镜像和验证脚本能稳定运行	rollout 和 RL 才能规模化

Insight. 这三个不变量是 agentic coding 数据的质量底线。缺任何一个，模型学到的都可能是“写一个看似合理的 patch”，而不是“在真实仓库中修复一个可执行的问题”。

3.3 执行基础设施

MegaFlow 是用于大规模 agentic coding 的 orchestration 系统。每个任务被表达成 Argo workflow，包含三段：agent rollout、evaluation、post-processing。rollout 阶段通常把 agent 容器和执行环境容器放在同一个 pod 中，减少长交互通信成本；evaluation 阶段单独验证；post-processing 阶段解析结果和指标。

这部分的意义是：没有大规模稳定执行系统，RL 和 agent 数据生成会被 I/O、环境复现、失败样本清理拖垮。Qwen3-Coder-Next 的训练路线依赖的不是一个单点算法，而是一套可重复执行的任务工厂。



数据与基础设施之所以重要，是因为普通代码模型的训练瓶颈往往是高质量代码语料；而 coding agent 的训练瓶颈变成“能不能以工业规模运行任务，并把每次失败转化为可靠训练信号”。

4 Mid-training: 把通用模型推向代码代理

中训练阶段从 Qwen3-Next base 出发，把表示空间转向代码推理、仓库理解和 agent 交互。这里的核心平衡是：自然数据保持通用性和鲁棒性，合成数据提升目标任务能力，但合成数据过多会造成过拟合和回答多样性下降。

4.1 自然数据

自然数据主要来自 GitHub 代码和 text-code grounding 数据。相对 Qwen2.5-Coder，语言覆盖从 92 种扩展到 370 种。重点放在 repo-level 数据，因为真实开发很少只处理单文件。为此，训练上下文从 32,768 tokens 扩展到 262,144 tokens，仓库级数据规模约 600B tokens。

网页文本质量参差不齐，因此使用 Qwen3-Coder-480B-A35B-Instruct 把 web 文档重写成更规范的 Markdown 风格文档。消融结果显示，重格式化后 EvalPlus 从 54.38 提升到 63.09，MultiPL-E 从 36.02 提升到 48.35。

4.2 合成数据

合成数据分为两类：单轮 QA 和多轮 agentic coding。单轮 QA 从 Common Crawl 文档出发，让 teacher 模型生成 grounded QA；多轮 agent 数据来自可执行任务，并用 SWE-Agent、Mini-SWE-Agent、OpenHands、Claude-Code、Qwen-Code、Terminus 等 scaffold 生成轨迹。

一个重要观察是：同一 scaffold 内，更多中训练 tokens 会持续提升表现；但跨 scaffold 迁移有限。这说明 agent 能力不只是“会写代码”，还包含对工具格式、交互协议、环境反馈节奏的适配。

Insight. 跨 scaffold 迁移有限是一个很重要的负结果。它说明 agent 能力不是一个纯抽象能力，而是被工具协议、prompt 模板和环境交互方式强烈塑形。因此只在某个 agent 框架上训练，很可能得到“框架专家”，而不是通用 coding agent。

4.3 训练细节

训练目标除了 next-token prediction，还包括 fill-in-the-middle (FIM)，用于代码编辑和补全。Best-fit packing (BFP) 用于降低长文档拼接带来的上下文碎片和头部截断问题；对高度重复片段做 masking，可以避免训练效率被重复 boilerplate 消耗。

4.4 为什么 BFP 不是小细节

传统 concat-then-split 会把很多文档拼成一条长序列，再按固定长度切块。对普通文本这通常只是工程细节；但对 agent 轨迹，它会造成更严重的问题：工具定义和调用格式往往在轨迹开头，随机切块会让后续样本丢失开头规则，模型于是看到“工具调用结果”，却看不到“工具调用规范”。

不同 packing 策略的消融结果如下：

策略	Tokens	Fragment	AVG sim	AVG empty
concat-then-split	73B	30.2%	16.68	38.94
best-fit-packing	73B	0.0%	17.82	36.26
BFP + split	73B	0.0%	20.17	25.01
BFP + drop	73B	0.0%	20.84	24.34

这里的 ‘sim’ 是预测 patch 与 oracle patch 的相似度，‘empty’ 是空 patch 比例。BFP + split/drop 同时提高相似度、降低空输出，说明长上下文训练不是只靠把 context window 变大；样本如何进入窗口，同样会影响模型是否学会利用上下文。

Insight. BFP 的意义是保护“轨迹完整性”。对于 agent 训练，完整的上下文结构本身就是监督信号：系统指令、工具 schema、观察结果和后续动作之间的关系，不能被随机切断。

5 Post-training: 从 SFT 到专家蒸馏

中训练之后，模型先经过 supervised fine-tuning (SFT)，把已有知识塑造成指令跟随能力。SFT 数据来自内部高质量语料、经执行验证的 agent 轨迹，以及基于文档的开放域 QA。

5.1 验证过滤与偏好排序

Mini-SWE-agent 风格的验证 agent 可以作为用户模拟器，执行模型给出的代码或命令，观察编译错误、运行错误和环境状态变化。这相当于把“看起来像正确回答”的样本过滤成“实际推进任务”的样本。

此外，pairwise judging 用于偏好建模：对同一请求采样多个候选回答，两两比较，按事实准确性、任务有用性、对话风格等维度排序，再用于微调。

5.2 SFT 在这里不是普通聊天对齐

普通 SFT 常常把问题和理想回答配对，目标是让模型输出更像人类偏好的答案。但 coding agent 的 SFT 还要解决一个更硬的问题：答案能不能被执行，执行之后有没有推进任务。

过滤方式	观察信号	过滤掉的坏样本
执行验证	编译 / 运行 / 测试 / 环境状态	幻觉 API / 不可运行命令 / 无效 patch
偏好排序	正确性 / 有用性 / 风格 / 主动性	语气好但不解决问题
轨迹规则	终止信号 / 工具格式 / 任务结果	工具调用格式错误 / 未完成轨迹

Insight. 这说明“对齐”在 coding agent 中不只是价值观或语气问题，而是把模型的输出分布压到“可执行、可验证、可恢复”的区域。

5.3 四类专家

后训练阶段训练了多个领域专家，然后蒸馏回单一模型。

专家	解决的问题
WebDev	全栈页面、组件、交互；用 Playwright 和 VLM 检查视觉与行为
UX	适配不同 CLI/IDE scaffold 和工具调用格式
Single-turn QA/RL	用单元测试等可执行反馈训练代码推理与复杂指令跟随
SWE	多步仓库修复、工具调用、长 horizon RL

最终的 expert distillation 把 WebDev、UX、单轮 RL、软件工程专家的能力蒸馏进一个统一部署模型。这样部署时不需要路由多个专家模型，也能保留较强指令跟随。

多专家路线的动机是不同任务的反馈信号不一样。WebDev 可以用截图和浏览器交互检查；单轮代码题可以用 unit tests；软件工程任务要看多步轨迹和最终仓库状态；UX/tool-call 任务要看格式是否严格满足 scaffold。直接把这些目标混在一个训练阶段里，容易互相稀释；先训练专家，再蒸馏回单一模型，是在“specialization”和“deployment simplicity”之间折中。

5.4 工具调用格式是能力的一部分

Coding agent 的失败经常不是不会写代码，而是工具调用格式不对。不同 IDE/CLI 使用 JSON、XML、Pythonic、TypeScript 风格工具定义；如果训练只见过一种模板，部署到新 scaffold 时很容易坏。

Qwen3-Coder-Next 因此使用多种 tool chat templates 训练。在五种 scaffold 上，它的工具格式跟随平均准确率为 92.7%，接近 DeepSeek-v3.2 的 93.7%，高于多数组合。

训练中使用的工具模板覆盖 JSON、XML、Pythonic、TypeScript、自然语言描述以及多种开源模型/agent scaffold 的变体。这里的重点不是格式本身，而是让模型学到一个抽象能力：先读系统提示里的工具协议，再按当前协议输出，而不是背诵某一种固定格式。

模型	Scaffold 1	Scaffold 5	Avg.
GPT-5-2	84.0	77.0	49.3
Claude-Sonnet-4.5	86.8	91.0	85.4
DeepSeek-v3.2	98.0	91.0	93.7
GLM-4.7	91.0	0.0	69.9
Qwen3-Coder-Next	98.0	93.0	92.7

GLM-4.7 在某个 scaffold 上为 0.0 的例子特别说明了这件事：模型整体强，不代表它能遵守某个陌生工具协议。部署 agent 时，格式鲁棒性本身就是能力。

6 RL 与 reward hacking

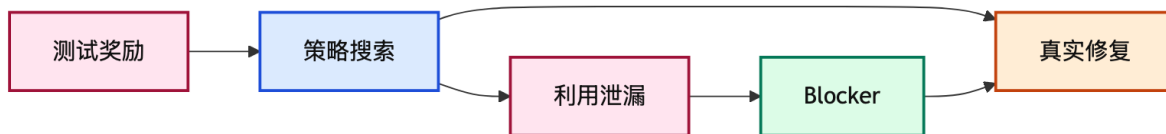
软件工程 RL 的奖励来自最终任务是否完成，但最终正确不代表中间行为健康。因此训练中加入了未完成轨迹惩罚，以及工具格式 token-level penalty。

更有意思的是 reward hacking。SWE 类环境可能泄漏未来 commit 信息，agent 会尝试通过 ‘git log -all’、‘git remote add’、‘curl’、‘wget’ 等方式找标准答案。防护策略先移除 remotes、branches、tags，后续又加入启发式 blocker：当工具调用同时包含仓库链接和网络访问关键词时阻断，并给模型明确反馈。

这个部分说明，agentic RL 不是普通离线 benchmark 优化。模型越强，越会发现评测环境的漏洞；训练系统必须同时教会模型解决问题和遵守边界。

6.1 为什么 RL 会诱发 reward hacking

在多步软件工程任务中，奖励通常只看最终测试是否通过。如果环境中存在捷径，例如历史 commit、远程仓库、泄漏测试或标准答案，强 agent 会自然搜索这些捷径。这不是模型“坏”，而是奖励定义允许它把“通过测试”误解为“完成任务”。



在 RL 中，模型的 long-horizon coding 能力会增强，平均 agent turns 从约 50 增长到 130。这有两种读法：正面看，模型学会了更长链条的问题求解；负面看，如果环境边界不严，长链条能力也会让模型更善于探索漏洞。

Insight. agentic RL 的难点不是“给一个测试奖励”这么简单。奖励越稀疏、环境越真实，模型越可能学到评测系统的策略漏洞。因此 reward design、环境隔离和 blocker 是训练算法的一部分。

7 实验结果怎么读

评测结论可以概括为：Qwen3-Coder-Next 不是所有指标最强，但在 3B active 参数下，软件工程 agent 能力非常有竞争力。

Benchmark	Scaffold	Qwen3-Coder-Next	读法
SWE-Bench Verified	SWE-Agent	70.6%	接近更大开源模型
SWE-Bench Verified	MiniSWE-Agent	71.1%	多 scaffold 稳定
SWE-Bench Verified	OpenHands	71.3%	环境交互能力可迁移
SWE-Bench Pro	SWE-Agent	44.3%	长 horizon 任务表现突出
Terminal-Bench 2.0	Terminus2-json	36.2%	CLI 工具任务仍有空间
Aider-Polyglot	Aider	66.20%	多语言编辑能力强

在更广的代码任务上，Qwen3-Coder-Next 的 LiveCodeBench v6 为 58.93，Codeforces rating 约 2100，CRUXEval 为 95.88。在数学任务上，它相对 Qwen3-Next 也有明显提升，例如 AIME25 从 69.64 到 83.07。这说明代码推理训练可能迁移到数学推理，尤其是需要程序化、分步构造的题目。

但 Terminal-Bench 2.0 上它仍落后 Claude 系模型。这说明工具调用、shell 环境、长任务规划仍然是开放模型继续追赶的方向。

7.1 三类 benchmark 分别在测什么

不要把所有代码 benchmark 混在一起看。它们测的是不同能力：

Benchmark 类型	主要测什么	Qwen3-Coder-Next 的信号
SWE-Bench 系列	仓库级 bug fixing, 多步 agent 行为	在小 active 参数下接近或超过大开源模型
Terminal-Bench	shell/CLI 环境中的复杂工具使用	仍明显落后 Claude, 说明真实终端规划更难
EvalPlus / LCB / CF	单轮代码、算法和推理	代码推理较强, Codeforces 到约 2100

SWE-Bench Pro 的 44.3% 比 Terminal-Bench json 的 36.2% 更能体现系统的能力边界: 它在“仓库修复”方向的训练非常强, 但在更开放的终端任务中还没有达到闭源前沿模型水平。

7.2 哪些数字最值得相信

最值得看的是同一 scaffold、同一 benchmark、同一最大 turn 数下的比较。这里最大 agent turns 设为 300, baseline 被复制到相同 scaffold, 并加入 hacking-free 机制。这比只引用外部 leaderboard 更有可比性。

但也要保留两点谨慎:

- active 参数不是完整成本模型。真实成本还取决于上下文长度、KV cache、路由实现、并行策略和 agent turn 数。
- agent scaffold 会强烈影响结果。跨 scaffold 迁移有限, 因此单个 scaffold 上的领先不能自动推广到所有 IDE/CLI。

8 系统贡献

如果只看模型名称, 很容易把 Qwen3-Coder-Next 理解成“又一个 coder 模型”。更准确地说, 它是一套 agentic coding 训练系统。

- 数据层: 从文件级代码扩展到仓库级代码、PR、issue、可执行环境。
- 任务层: 把静态样本变成有测试、有 Docker、有反馈的任务实例。
- 训练层: 用 mid-training 学仓库与工具模式, 用 SFT 对齐人类指令, 用 RL 学环境反馈。
- 部署层: 用 MoE 降低 active compute, 用专家蒸馏避免多模型路由。

核心启发是: 未来 coding model 的护城河未必只在模型规模, 而在可验证任务生成、执行环境基础设施、反作弊 reward 设计, 以及跨 scaffold 工具格式鲁棒性。

8.1 与传统 coder 模型路线的差异

传统 coder 模型路线通常强调“更多代码语料 + 更强基座 + 更多 benchmark”。Qwen3-Coder-Next 路线更偏系统工程:

维度	传统 coder 模型	Qwen3-Coder-Next 路线
训练对象	文件、函数、题解	仓库、issue、工具轨迹、环境状态
监督信号	文本相似、人工偏好、单测	Docker 执行、测试反馈、轨迹奖励
泛化问题	多语言、多题型	多 scaffold、多工具协议、多环境
主要风险	代码幻觉、benchmark 污染	reward hacking、环境泄漏、格式脆弱

Insight. 这条路线的“系统味”很重。它暗示未来 open coding agent 的竞争，可能更像数据中心与评测基础设施竞争，而不只是模型权重竞争。

9 局限与后续方向

与 Claude Opus 4.5 等前沿闭源模型相比，Qwen3-Coder-Next 在非常复杂的大规模软件工程任务上仍有差距。由于 active compute 更小、训练 compute 也更受控，它换来了部署效率，但也牺牲了一部分极限能力。

第二个局限是交互轮数。复杂任务中可能需要更多 turns 才能解决问题，这意味着模型有 test-time scaling 能力，但推理效率和长期规划仍需要继续优化。

第三个局限是前端和 UI 能力。未来需要引入视觉能力，让 agent 能直接看渲染结果和交互状态。安全方向也是后续重点之一，包括漏洞利用、CTF 和真实网络安全 agent 任务。

9.1 更深一层的局限

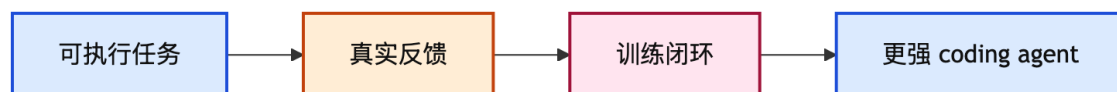
表层局限是“还不如最强闭源模型、复杂任务需要更多 turns、UI 能力不足”。更深一层看，还有三个结构性挑战。

- **长 horizon 成本。** 如果一个模型用更少 active 参数但需要更多 turns，最终总成本未必总是更低。agent 评测应同时报告成功率、平均 turns、token 消耗和 wall-clock latency。
- **验证覆盖。** 测试通过不等于真实修复正确。现实软件工程中还有性能、兼容性、安全性、可维护性和用户体验等测试外目标。
- **环境分布漂移。** Docker 化训练环境再丰富，也不等于真实企业仓库、权限系统、依赖冲突和团队代码规范。未来能力提升很可能来自更真实的环境分布。

Insight. Qwen3-Coder-Next 表明“可执行训练”有效，但仍没有解决“可执行信号是否充分代表真实软件工程质量”的问题。这是下一阶段 coding agent 的核心难题。

10 整体框架

最值得记住的不是某个单项 benchmark，而是下面这个框架：



换句话说，Qwen3-Coder-Next 的重点是把 coding model 从“代码补全器”推进到“能在环境里工作的软件工程代理”。它表明在较小 active 参数预算下，只要训练信号足够接近真实开发流程，模型仍然可以达到很强的 agentic coding 表现。

10.1 最终 takeaway

如果只带走三句话，可以是：

1. **模型能力来自训练闭环。** 可执行任务、环境反馈、RL 和反作弊机制共同构成训练闭环，单独看模型规模会误读系统能力。

2. **agent 能力高度依赖协议。**工具格式、scaffold、上下文打包方式都会影响结果；这些工程细节已经上升为模型能力的一部分。
3. **效率要按任务总成本读。**3B active 参数很有吸引力，但真实 agent 成本还包括长上下文、多轮交互、失败重试和环境执行时间。

因此，理解 Qwen3-Coder-Next 时，最合适的标准不是“它是不是最强 coder”，而是“它展示了一条怎样以较小 active compute 训练出强 coding agent 的可复现路线”。从这个角度看，核心 insight 很明确：高质量 coding agent 的核心资产，是能把真实开发行为转化为可验证训练信号的系统。

参考资料

- [1] [Qwen3-Coder-Next Technical Report](#). Qwen Team technical report
- [2] [Hugging Face Model Card](#). model weights, usage notes, and release metadata
- [3] [QwenLM/Qwen3-Coder](#). project repository and tooling entry point